

## Module 02: Linked List, Stack, Queue

**Language:** Java**Type:** Unguided Lab**Mode:** individual practical work

---

### Learning outcomes

**By completing this module, you should be able to:**

1. Use LinkedList to store and manage a collection of data.
2. Use Queue to process data according to the FIFO (First In First Out) principle.
3. Use Stack to store and retrieve data according to the LIFO (Last In First Out) principle.
4. Read structured data from a text file using Scanner.
5. Process data sequentially by combining LinkedList, Queue, and Stack.
6. Explain how each data structure is used to solve a specific part of a problem.

### Concept Toolkit

| Concept    | Meaning and a useful distinction                                                                                                                   |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| LinkedList | A linear data structure whose elements are connected sequentially. It can be used to store, add, remove, and access data in an ordered collection. |
| Queue      | A data structure that follows the FIFO (First In First Out) principle. The element that enters first is processed first.                           |
| Stack      | A data structure that follows the LIFO (Last In First Out) principle. The element added most recently is retrieved first.                          |
| poll()     | Removes and returns the element at the front of a queue. It follows the FIFO principle. If the queue is empty, poll() returns null.                |
| push()     | Adds an element to the top of a stack. The newly added element becomes the first element to be retrieved by pop().                                 |
| pop()      | Removes and returns the element at the top of a stack. It follows the LIFO principle.                                                              |
| peek()     | Returns the element at the front of a queue or the top of a stack without removing it.                                                             |

## Unguided Case: Library Book Borrowing

A campus library wants a simple program to handle book borrowing requests. All requests are listed in a text file called `borrowing.txt`. Each line shows the **borrower's name** and the **title of the book** they want to borrow.

Each book title has a **limited stock**. Each member can also borrow only a **limited number of books** at the same time. Once a member reaches this limit, their next request **cannot be processed**, even if the book they want is still in stock.

The program processes requests in the same order they appear in the file. For each request, it checks **two things**:

1. Does the book still **have stock**?
2. Has the member reached their **borrowing limit**?

A request only succeeds if the answer to both is **yes**. If either answer is **no**, the request fails and is saved as a **failed request**.

The program must use different data structures for different purposes:

- A **LinkedList** stores **every request** from the file.
- Another **LinkedList** stores **each book** and its **current stock**.
- A third **LinkedList** stores **each member** and **how many books they have borrowed**.
- A **Queue** processes the requests one by one, in the order they arrived (**FIFO**).
- A **Stack** stores **every request that fails**, whether because the book is out of stock or the member reached their limit.

***Note:** You do not need to create separate `Request`, `Book`, or `Member` classes. Use simple structures such as `String[]` so that the main focus remains on understanding `LinkedList`, `Queue`, and `Stack`.*

After all requests are processed, the program shows: the **requests that succeeded**, the **remaining stock** of each book, and the **requests that failed**. Failed requests are shown starting from the most recent failure.

### Exact Behaviour

- Read the borrowing data from `borrowing.txt` using Scanner.
- Each line has two values, separated by a space: `<NAME> <BOOK_TITLE>`.
- `<NAME>` is the borrower's name.
- `<BOOK_TITLE>` is the title of the requested book.
- Store every request in a LinkedList, in the same order as the file.
- Store each book and its starting stock in a separate LinkedList. Starting stock:
  - Kalkulus: 2
  - Fisika: 1
  - Statistika: 2

#### Labwork 02

- Store each member in a separate LinkedList. Add a member the first time their name appears, in that order. Every member starts with 0 borrowed books.
- Each member can borrow at most **2** books at a time (**MAX\_BORROW = 2**).
- Move all requests from the LinkedList into a Queue.
- Process the requests in FIFO order.
- For each request:
  - Check if the book still has stock and if the member has not reached the limit.
  - If both are true, the request succeeds: save it as a successful request, reduce the book's stock by 1, and increase the member's borrowed count by 1.
  - If either is false, the request fails: push it onto the Stack.
- Keep going until the Queue is empty.
- Show the successful requests, in the order they were processed.
- Show the remaining stock of every book.
- Pop the failed requests from the Stack and show them in LIFO order (most recent failure first).

#### Unguided Tasks

##### A1. Implementation and demonstration

**Note: Only use the methods we have already learned. Do not use other built-in methods that have not been covered in this module.**

Create a Java program that processes library book borrowing requests using **LinkedList**, **Queue**, and **Stack** following these steps:

1. **Read and store requests:** Create a LinkedList to store all borrowing requests. Read **borrowing.txt** using Scanner. For each line, parse the borrower's name and the requested book title, and store it as a **String[]**, preserving the original order.
2. **Create book and member data:** Create another LinkedList to store book records (title and current stock), initialized with the starting stock (Kalkulus: 2, Fisika: 1, Statistika: 2). Create a third LinkedList to store member records (name and borrowed book count). Set every member's initial borrowed count to 0. Add a member to this list only the first time their name appears in the request data, preserving that first-appearance order. Do not add the same member twice.
3. **Process requests using Queue:** Create a Queue and move all requests from the request LinkedList into it. Process the queue in FIFO order: for each request, find the corresponding book and member, then check whether the book still has stock and whether the member has reached the borrowing limit (**MAX\_BORROW = 2**). If both conditions allow it, the request succeeds, update the book's stock and the member's borrowed count, and store the request as a successful request.
4. **Store failed requests using Stack:** Create a Stack. Whenever a request fails, because the book is out of stock, because the member has reached the borrowing limit, or

## Labwork 02

both, push it onto the stack. After all requests are processed, retrieve and display the failed requests in LIFO order (as produced by the stack).

- Finally, **display the successful requests** in the order they were processed, the **remaining stock of every book**, and **every failed request**, matching the required output format.

Input file `borrowing.txt`:

Farel Kalkulus

Farel Fisika

Farel Statistika

Taqiy Statistika

Michelle Fisika

Taqiy Kalkulus

Michelle Statistika

Raissa Statistika

## A2. Required checks

| Request             | Book Stock Before | Member Status Before | Result  | Book Stock After | Member Status After |
|---------------------|-------------------|----------------------|---------|------------------|---------------------|
| Farel Kalkulus      | Kalkulus: 2       | Farel: 0/2           | Success | Kalkulus: 1      | Farel: 1/2          |
| Farel Fisika        | Fisika: 1         | Farel: 1/2           | Success | Fisika: 0        | Farel: 2/2          |
| Farel Statistika    | Statistika: 2     | Farel: 2/2           | Failed  | Statistika: 2    | Farel: 2/2          |
| Taqiy Statistika    | Statistika: 2     | Taqiy: 0/2           | Success | Statistika: 1    | Taqiy: 1/2          |
| Michelle Fisika     | Fisika: 0         | Michelle: 0/2        | Failed  | Fisika: 0        | Michelle: 0/2       |
| Taqiy Kalkulus      | Kalkulus: 1       | Taqiy: 1/2           | Success | Kalkulus: 0      | Taqiy: 2/2          |
| Michelle Statistika | Statistika: 1     | Michelle: 0/2        | Success | Statistika: 0    | Michelle: 1/2       |
| Raissa Statistika   | Statistika: 0     | Raissa: 0/2          | Failed  | Statistika: 0    | Raissa: 0/2         |

### Labwork 02

#### Program output:

=== Successfully Processed Requests ===

Farel Kalkulus

Farel Fisika

Taqiy Statistika

Taqiy Kalkulus

Michelle Statistika

=== Remaining Book Stock ===

Kalkulus : 0

Fisika : 0

Statistika : 0

=== Failed Requests ===

Raissa Statistika

Michelle Fisika

Farel Statistika

#### Submission

Please follow these instructions carefully to submit your work:

1. Inside the previously created **lw02** folder under your **dsa-[nrp]** repository, create a subfolder named **unguided**.
2. Complete all of your work for this assessed lab inside the **unguided** folder.
3. Commit and push your work to GitHub with the commit message:  
"Labwork 02 - Unguided"

To give a clearer picture, The repository for this semester's practicum will follow the structure below:

```
dsa-[nrp]/
├── src/
│   ├── lw01/
│   │   ├── prelab/
│   │   │   └── [your prelab files]
│   │   └── unguided/
│   │       └── [your unguided files]
│   ├── lw02/
│   │   ├── prelab/
│   │   │   └── [your prelab files]
│   │   └── unguided/
│   │       └── [your unguided files]
│   ├── lw03/
│   │   ├── prelab/
│   │   │   └── [your prelab files]
│   │   └── unguided/
│   │       └── [your unguided files]
│   └── ...
```